CSS Cheatsheet

This is everything from the CSS Lecture. CSS (**Cascading Style Sheets**) is a list of rules for how a page should look. When more than one rule could apply to an element, the more specific or later rule wins.

Anatomy of a Rule

```
selector {
  property: value;
  /* ... */
}
```

Part	What it is	Example
selector	which elements the rule applies to	`p`, `.compact`, `#about`
property	what you are changing	`color`, `font-size`
value	what you are changing it to	`red`, `48px`
`;`	ends each `property: value` pair	
`/* ... */`	a comment; the browser ignores it	`/* header styles */`

Three Places for CSS

Kind	Where it goes	Use when
Inline	`style` attribute on one element	A one-off tweak to a single element
Internal	`<style>` in the `<head>` (also called embedded)	Rules for this one page
External	`<link rel="stylesheet" href="style.css" />`	Rules shared by every page on the site

```
<!-- Inline -->
<p style="color: red;">This paragraph is red.</p>

<!-- Internal -->
<head>
  <style>
    p {
      color: red;
    }
  </style>
</head>

<!-- External: index.html -->
<head>
  <link rel="stylesheet" href="style.css" />
</head>
```

```css
/* External: style.css */
p {
  color: red;
}
```

Inline beats internal and external, unless a rule says `!important`. Internal and external are equals: neither wins just for being where it is. When an internal rule and an external rule conflict, the more specific selector wins, and if they are equally specific, whichever appears later in the `<head>` wins. Both ideas are spelled out in Which Rule Wins.

Selectors

The Basics

Selector	Syntax	Matches	Use when
Tag	`p`	every `<p>`	Every element of that type should look the same
Class	`.favorite`	every element with `class="favorite"`	A group of elements share a look
ID	`#header`	the one element with `id="header"`	Exactly one element needs its own look
Attribute	`a[href]`	every `<a>` that has an `href` attribute	Elements that have (or lack) a given attribute

Focus on the selector in each rule below. The properties inside the braces are covered in Properties from the Lecture.

```css
/* every <p> */
p {
  color: red;
}

/* every <img> */
img {
  opacity: 0.5;
}

/* every <td> (a table cell) */
td {
  text-align: center;
}

/* everything with class="favorite" */
.favorite {
  font-weight: bold;
}

/* the one element with id="header" */
#header {
  text-transform: uppercase;
}
```

```
<p class="favorite">A class can go on as many elements as you like.</p>
<p class="favorite compact">
  An element can have several classes, separated by spaces.
</p>
<h2 id="header">An id must be unique: one element per name on the page.</h2>
```

`.` selects by class; `#` selects by ID. `id` is also what `<a href="#header">` jumps to.

Class Versus ID

Question	Class	ID
HTML attribute	`class="about"`	`id="about"`
CSS selector	`.about`	`#about`
How many elements can have it?	as many as you like	exactly one per page
Can one element have several?	yes: `class="bold italicized"`	no
Specificity	lower	higher: ID beats class
Also used for		`<a href="#about">` jump links

Giving two elements the same `id` is invalid HTML. The browser will not stop you: `<a href="#about">` jumps to the first one, and `#about { }` styles both.

Combining Selectors

Pattern	Meaning	Example	Matches
No space	**and**	`span.bold`	a `<span>` that also has class `bold`
Space	**inside**	`p .bold`	anything with class `bold` that is inside a `<p>`
Comma	**each of these**	`span, a`	every `<span>` and every `<a>`
Colon	**in this state**	`a:hover`	a link the mouse is over
Double colon	**this part of**	`p::first-letter`	the first letter of every `<p>`
Square brackets	**has attribute**	`img[alt]`	an image with an `alt` attribute

Watch the space. `p.bold` is a `<p>` with class `bold`. `p .bold` is an element with class `bold` inside a `<p>`.

```css
/* every span, every link, and everything with class blackAndYellow */
span,
a,
.blackAndYellow {
  background-color: black;
  color: yellow;
}

/* links inside a paragraph, while the mouse is over them */
p a:hover {
  font-size: 48px;
}

/* visited links inside the reviews list only */
.reviews a:visited {
  color: gray;
}
```

Attribute Selectors

`selector[attribute]`. Works for any attribute: `alt`, `href`, `target`, `class`, `lang`, … Full reference on MDN.

Selector	Matches when the attribute…	Example
`[attr]`	exists	`img[alt]`
`[attr="v"]`	is exactly `v`	`a[target="_blank"]`
`[attr^="v"]`	starts with `v`	`a[href^="https"]`
`[attr$="v"]`	ends with `v`	`a[href$=".pdf"]`
`[attr*="v"]`	contains `v` anywhere	`a[href*="google"]`
`[attr~="v"]`	is a space-separated list containing `v`	`p[class~="compact"]`
`[attr|="v"]`	is `v` or starts with `v-`	`p[lang|="en"]`

```css
/* links that open in a new tab */
a[target="_blank"] {
  letter-spacing: 5px;
}
```

Pseudo-Classes: Selecting by State

`selector:state`. A pseudo-class picks elements by what state they are in, and counts like a class when deciding which rule wins. Full list on MDN.

Pseudo-class	Matches when
`:hover`	the mouse is over the element
`:visited`	the user has already been to the link's address
`:active`	the mouse button is held down on the element
`:not(X)`	the element does **not** match `X`

```
/* select a link, while the mouse is over it */
a:hover {
  color: orange;
}

/* select an image that does not have an alt attribute */
img:not([alt]) {
  opacity: 0.2;
}
```

`:visited` can only change colors. Browsers block anything else for privacy.

Order matters. `a:hover` and `a:visited` are equally specific, so if a link is both hovered and visited, whichever rule is further down wins.

Pseudo-Elements: Selecting Part of an Element

`selector::part`. Two colons. Full list on MDN.

Pseudo-element	Selects
`::first-letter`	the first letter of the element
`::first-line`	the first line (depends on the window width)
`::selection`	the text the user has highlighted
`::before`, `::after`	content added before or after the element

```
p::first-letter {
  font-size: 48px;
}

p::first-line {
  text-transform: uppercase;
}
```

Inheritance

- An element inside another element is its **child**; the element it sits inside is its **parent**. `<li>` is a child of `<ul>`, which is a child of `<body>`, which is a child of `<html>`.
- When a rule sets a property on an element, its children start out with the same value. Style `<body>` and everything on the page matches.

- A child only uses the inherited value when no rule sets that property on it directly. If a rule sets the property on the child itself, that rule wins.
- Text and color properties inherit (`color`, `font-size`, `font-family`, `line-height`). Box properties do not (`border`, `margin`, `padding`, `width`). When in doubt, every MDN property page has an **Inherited: yes/no** row.

```
body {
  color: red;
}
div {
  color: purple;
}
```

Everything is red, except text inside a `<div>`, which is purple.

The Browser's Default Stylesheet

Every browser ships with its own stylesheet (the **user agent stylesheet**). It is why `<h1>` is big, `<a>` is blue and underlined, and `<ul>` has bullets before you write any CSS. A simplified piece of it:

```
h1 {
  font-size: 2em;
}
a {
  color: blue;
  text-decoration: underline;
}
ul {
  list-style: disc;
  padding-left: 40px;
}
```

- The browser's stylesheet applies first; yours is layered after it.
- If one of your rules sets a property on an element, it beats the browser's rule for that property.
- `body { color: red; }` does not turn links red: the browser's `a` rule matches the link directly, and a direct match beats inheritance. To change a default, write a rule that matches the element: `a { color: red; }`.

Which Rule Wins

More than one rule can match the same element. CSS resolves each property separately, in this order:

1. `!important` beats everything else.
2. **Your rules beat the browser's.** The browser's default stylesheet only applies where none of your rules set the property.
3. **Inline** (`style="..."`) beats internal and external.
4. **The most specific selector wins: ID > class > tag.** Count each kind separately. More IDs wins; on a tie, more classes (pseudo-classes and `[attribute]` selectors count as classes); on a tie, more tags. So `span a` (two tags) beats `a` (one tag), but `.about` (one class) beats `span a`.

5. **On a tie, the rule further down in the CSS wins.**
6. **Inheritance is the fallback.** If no rule sets the property on an element at all, it uses its parent's value (for properties that inherit).

```
p {
  color: black;
}
.note {
  color: blue;
}
#warning {
  color: red;
}
```

```
<p class="note" id="warning">Red: the ID wins.</p>
```

```
p {
  font-size: 16px;
}
```

```
p {
  font-size: 48px;
}
```

```
<p>48px: same selector, so the later rule wins.</p>
```

Specificity Examples

Each row beats every row above it.

Selector	IDs	Classes	Tags	Beats the row above because
`a`	0	0	1	
`span a`	0	0	2	two tags beat one
`.about`	0	1	0	one class beats any number of tags
`a:hover`	0	1	1	same classes, one more tag
`p a:hover`	0	1	2	same classes, one more tag
`.reviews a:visited`	0	2	1	two classes beat one
`#about`	1	0	0	one ID beats any number of classes

`!important`

```
p {
  color: red !important;
}
```

Overrides inline CSS and every other rule for that property. Be careful with this. Only use it when you feel like you have no other choices.

Values

The right-hand side of `property: value`. Two kinds show up in almost every rule: colors and lengths.

Colors

Form	Example	Notes
Named	`red`, `rebeccapurple`	148 of them. MDN lists them all.
`rgb()`	`rgb(201, 123, 58)`	red, green, blue: how much of each, from `0` (none) to `255` (full)
Hex	`#c97b3a`	same idea as `rgb()`, two hex digits per channel
`hsl()`	`hsl(27, 57%, 51%)`	hue, saturation, lightness

`rgb()`	Color
`rgb(255, 0, 0)`	red
`rgb(0, 255, 0)`	green
`rgb(0, 0, 255)`	blue
`rgb(0, 0, 0)`	black
`rgb(128, 128, 128)`	gray
`rgb(255, 255, 255)`	white

Units

Unit	Meaning	Example
`px`	pixels	`font-size: 48px`
`%`	a percentage of the parent's value	`font-size: 150%`
(none)	a plain number; for `line-height`, a multiple of the font size	`line-height: 2`

Properties from the Lecture

There are hundreds of properties. Learn the pattern (`selector { property: value; }`), then look them up in the MDN CSS reference.

Color and Background

Property	Values	What it does
`color`	any color	the text color
`background-color`	any color	the color of the box behind it
`opacity`	`0` (invisible) to `1` (solid)	how see-through the whole element is

Fonts

Property	Values	What it does
`font-family`	a list: `Georgia, serif`	which typeface
`font-size`	`16px`, `150%`	how big the letters are
`font-weight`	`normal`, `bold`, `100` to `900`	how heavy the letters are
`font-style`	`normal`, `italic`	whether the letters slant

`font-family` takes a list. The browser uses the first font it has installed. End the list with a generic family, which every browser has:

Generic family	Look	Examples
`serif`	letters with little feet	Georgia, Times New Roman
`sans-serif`	no feet	Arial, Helvetica
`monospace`	every letter the same width	Courier New

```
i {
  font-size: 150%;
  font-family: Georgia, serif;
}
```

Text

Property	Values	What it does
`text-align`	`left`, `center`, `right`	where the text sits in its box
`text-transform`	`uppercase`, `lowercase`, `capitalize`	changes the case
`text-decoration`	`none`, `underline`, `line-through`	lines on the text; `none` removes a link's underline
`letter-spacing`	a length, e.g. `2px`	space between letters
`line-height`	a number, e.g. `2`	space between lines, as a multiple of the font size

```
h2 {
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 2px;
}
p {
  line-height: 2;
}
```

Spacing

Property	Values	What it does
`padding`	a length, `50px`	the space between an element's text and the edge of its box

Recipes from the Lecture

Problem:	**CSS**
Every paragraph red	`p { color: red; }`
All text on the page blue	`body { color: blue; }`
All text red except inside `<div>`, which is purple	`body { color: red; } div { color: purple; }`
Every link with a yellow background	`a { background-color: yellow; }`
Every image half transparent	`img { opacity: 0.5; }`
Some paragraphs with less padding	`p { padding: 100px; } .compact { padding: 50px; }` and `class="compact"` on those paragraphs
A class that only works on `<span>`	`span.bold { font-weight: bold; }`
Every `<span>`, every `<a>`, and every `.blackAndYellow`	`span, a, .blackAndYellow { ... }`
One specific link with no underline	`#about { text-decoration: none; }` and `id="about"` on the link
A heading centered and in all caps	`h2 { text-align: center; text-transform: uppercase; }`
Links inside a paragraph grow on hover	`p a:hover { font-size: 48px; }`
Visited links in the reviews list turn gray	`.reviews a:visited { color: gray; }`
One specific link turns red while clicked	`.about:active { color: red; }` and `class="about"` on the link
Big first letter on every paragraph	`p::first-letter { font-size: 48px; }`
Uppercase first line of every paragraph	`p::first-line { text-transform: uppercase; }`
Only links that actually have an `href`	`a[href] { ... }`
Images missing `alt` text fade out	`img:not([alt]) { opacity: 0.2; }`
Links that open in a new tab	`a[target="_blank"] { ... }`
Make every paragraph 48px, overriding an earlier rule	Put the `48px` rule further down
Force a rule to win no matter what	`color: red !important;`

Vocabulary

Term	Meaning
Rule	a selector plus its `{ property: value; }` block
Selector	the part of a rule that picks which elements it applies to
Property	the thing being styled (`color`, `font-size`)
Value	what the property is set to (`red`, `48px`)
Inline / internal / external	CSS in a `style` attribute / in `<style>` / in a linked `.css` file
Cascade	the process of deciding which rule wins when several match
Specificity	how precisely a selector targets an element: ID > class > tag
Inheritance	children start with their parent's value for text and color properties
Parent / child	the element an element sits inside / an element that sits inside another
Attribute	a `name="value"` pair in an opening tag: `href`, `src`, `alt`, `class`, `id`, `style`
Pseudo-class	a selector for an element's state: `:hover`, `:visited`, `:active`, `:not()`
Pseudo-element	a selector for part of an element: `::first-letter`, `::first-line`
User agent stylesheet	the browser's own default CSS, applied before yours